Técnicas para el desarrollo de FrontEnd

Laboratorio de React

Componentes reutilizables: Login y Dashboards por rol

Dato	Detalle
Tema	Componentes, props, estado y renderizado condicional
Modalidad	Práctica individual en laboratorio
Duración estimada	4 horas
Herramientas	Node.js, Vite, React, editor Antigravity
Nombre del alumno	
Fecha de entrega	

1. Objetivos

Al terminar este laboratorio el alumno será capaz de:

- Diseñar una aplicación de React dividida en **componentes pequeños y reutilizables**.
- Comunicar datos entre componentes mediante **props**, incluyendo funciones (callbacks) y `children`.
- Manejar el **estado** con `useState` para formularios, sesión del usuario y navegación interna.
- Usar **renderizado condicional** para mostrar pantallas distintas según el rol del usuario.
- Organizar un proyecto con una **estructura de carpetas** clara y profesional.

2. Requisitos previos

- Node.js 20 o superior instalado. Verifica con `node -v` en la terminal.
- Editor Antigravity (o cualquier editor de código).
- Haber realizado la práctica anterior con `useState` e `input` controlado.

> **¿Por qué Vite y no create-react-app?**
> El equipo de React deprecó oficialmente **create-react-app** en 2025: ya no recibe mantenimiento y es mucho más lento. **Vite** es la herramienta que se usa actualmente para crear proyectos de React.
>
> Todo el código de este laboratorio también funciona en un proyecto creado con create-react-app. Las únicas diferencias son que el archivo principal se llama `App.js` y el servidor se inicia con `npm start`.

3. ¿Qué vamos a construir?

Un **Portal Escolar** con tres pantallas:

1. **Login:** formulario que valida correo y contraseña contra una lista de usuarios simulados.
2. **Dashboard de Administrador:** resumen con estadísticas, tabla de usuarios y tabla de cursos.
3. **Dashboard de Usuario:** inicio con su avance, lista de sus cursos y su perfil.

La idea central es que **los dos dashboards comparten los mismos componentes** (barra superior, menú lateral, tarjetas de estadísticas, tabla). Solo cambian los datos que reciben por props. Así se escribe el código una vez y se reutiliza.

3.1 Árbol de componentes

El siguiente árbol muestra qué componente contiene a cuál:

```
App  (guarda el usuario en sesión)
├── LoginPage
│   └── Card
│       ├── InputField  (correo)
│       ├── InputField  (contraseña)
│       └── Button
├── AdminDashboard
│   └── DashboardLayout
│       ├── Navbar ── Button
│       ├── Sidebar
│       └── contenido: StatCard, Card, DataTable
└── UserDashboard
    └── DashboardLayout   (¡el mismo que usa el admin!)
        ├── Navbar ── Button
        ├── Sidebar
        └── contenido: StatCard, Card, DataTable
```

3.2 Clasificación de los componentes

Tipo	Componentes	Responsabilidad
Comunes (common)	Button, InputField, Card	Piezas básicas de interfaz que se usan en cualquier parte.
Estructura (layout)	Navbar, Sidebar, DashboardLayout	Definen el “esqueleto” compartido por los dashboards.
Dashboard	StatCard, DataTable	Muestran información: números y tablas.
Páginas (pages)	LoginPage, AdminDashboard, UserDashboard	Combinan componentes y manejan su propio estado.

4. Parte 1: Crear el proyecto

Crea una carpeta llamada `portal-escolar`, ábrela en Antigravity y en la terminal ejecuta:

```
npm create vite@latest . -- --template react
npm install
npm run dev
```

El punto (`.`) indica que el proyecto se crea dentro de la carpeta actual, igual que con create-react-app. Si el asistente hace preguntas adicionales, acepta las opciones por defecto.

Abre en el navegador la dirección que aparece en la terminal (normalmente `http://localhost:5173`). Deberías ver la página de bienvenida de Vite.

4.1 Limpieza inicial

1. Elimina el archivo `src/App.css` y la carpeta `src/assets`.
2. Borra todo el contenido de `src/index.css` (lo llenaremos en la Parte 8).
3. No modifiques `src/main.jsx`: ya importa `index.css` y `App.jsx`.

5. Parte 2: Estructura de carpetas

Dentro de `src` crea las siguientes carpetas y archivos vacíos. Una buena estructura permite encontrar cualquier componente en segundos, aunque el proyecto crezca.

```
src/
├── components/
│   ├── common/
│   │   ├── Button.jsx
│   │   ├── Card.jsx
│   │   └── InputField.jsx
│   ├── layout/
│   │   ├── DashboardLayout.jsx
│   │   ├── Navbar.jsx
│   │   └── Sidebar.jsx
│   └── dashboard/
│       ├── DataTable.jsx
│       └── StatCard.jsx
├── data/
│   └── users.js
├── pages/
│   ├── AdminDashboard.jsx
│   ├── LoginPage.jsx
│   └── UserDashboard.jsx
├── App.jsx
├── index.css
└── main.jsx
```

> **Convención de nombres**
> Los archivos de componentes usan **PascalCase** (`Button.jsx`, `StatCard.jsx`) y extensión `.jsx`. Los archivos que solo contienen datos o funciones usan **camelCase** y extensión `.js` (`users.js`).

6. Parte 3: Datos simulados

Todavía no tenemos un servidor, así que simularemos la base de datos con arreglos de JavaScript. La palabra `export` permite importarlos desde otros archivos.

src/data/users.js

```
// Datos simulados: en un proyecto real vendrían de una API
export const users = [
  { id: 1, name: 'Ana Torres', email: 'admin@escuela.edu', password: 'admin123', role: 'admin'
},
  { id: 2, name: 'Luis Pérez', email: 'luis@escuela.edu', password: 'user123', role: 'user' },
  { id: 3, name: 'María López', email: 'maria@escuela.edu', password: 'user123', role: 'user'
},
];

export const courses = [
  { id: 1, name: 'Programación FrontEnd', progress: 75 },
  { id: 2, name: 'Bases de Datos', progress: 40 },
  { id: 3, name: 'Redes de Computadoras', progress: 90 },
];
```

> ⚠️ **Nota de seguridad**
>
> Guardar contraseñas en el código del FrontEnd **solo es aceptable en una práctica**. Cualquier persona puede verlas desde las herramientas de desarrollador del navegador. En una aplicación real, la validación se hace en el servidor (BackEnd) y las contraseñas se guardan cifradas con un hash.

7. Parte 4: Componentes comunes

Estos son los “ladrillos” de la aplicación. No saben nada del login ni de los dashboards: solo reciben props y las muestran.

7.1 Button

Un botón con variantes de color. Observa los **valores por defecto** en las props: si no se envía `variant`, será `"primary"`.

src/components/common/Button.jsx

```
const Button = ({ children, onClick, type = 'button', variant = 'primary', fullWidth = false
}) => {
  const classes = `btn btn-${variant} ${fullWidth ? 'btn-full' : ''}`;

  return (
    <button type={type} className={classes} onClick={onClick}>
      {children}
    </button>
  );
};

export default Button;
```

La prop especial `children` contiene lo que se escribe entre las etiquetas de apertura y cierre. Por ejemplo, en `<Button>Entrar</Button>`, children vale `"Entrar"`.

7.2 InputField

Agrupa una etiqueta y un campo de texto. Es un **input controlado**: su valor viene del estado del componente padre.

src/components/common/InputField.jsx

```jsx
const InputField = ({ label, type = 'text', name, value, onChange, placeholder }) => {
  return (
    <div className="input-field">
      <label htmlFor={name}>{label}</label>
      <input
        id={name}
        name={name}
        type={type}
        value={value}
        onChange={onChange}
        placeholder={placeholder}
      />
    </div>
  );
};

export default InputField;
```

En JSX se usa `htmlFor` en lugar de `for`, porque `for` es una palabra reservada de JavaScript. Al conectar `htmlFor` con el `id` del input, al hacer clic en la etiqueta se activa el campo.

7.3 Card

Un contenedor con fondo blanco y sombra. El título es opcional: la expresión `title && ...` solo muestra el `h3` si se envió un título.

src/components/common/Card.jsx

```jsx
const Card = ({ title, children }) => {
  return (
    <div className="card">
      {title && <h3 className="card-title">{title}</h3>}
      {children}
    </div>
  );
};

export default Card;
```

8. Parte 5: Componentes de estructura (layout)

8.1 Navbar

Barra superior con el nombre del usuario y el botón para cerrar sesión. Observa que **reutiliza** el componente `Button` con la variante `secondary`.

src/components/layout/Navbar.jsx

```jsx
import Button from '../common/Button';

const Navbar = ({ user, onLogout }) => {
  const roleLabel = user.role === 'admin' ? 'Administrador' : 'Usuario';

  return (
    <header className="navbar">
      <span className="navbar-brand">Portal Escolar</span>
      <div className="navbar-user">
        <span>{user.name} ({roleLabel})</span>
        <Button variant="secondary" onClick={onLogout}>
          Cerrar sesión
        </Button>
      </div>
    </header>
  );
};

export default Navbar;
```

8.2 Sidebar

Menú lateral generado a partir de un arreglo con `map()`. Cada elemento de una lista necesita una prop `key` única para que React pueda identificarlo.

src/components/layout/Sidebar.jsx

```jsx
const Sidebar = ({ items, activeItem, onSelect }) => {
  return (
    <aside className="sidebar">
      <ul>
        {items.map((item) => (
          <li key={item}>
            <button
              className={item === activeItem ? 'active' : ''}
              onClick={() => onSelect(item)}
            >
              {item}
            </button>
          </li>
        ))}
      </ul>
    </aside>
  );
};

export default Sidebar;
```

Nota que el Sidebar **no guarda estado**: recibe cuál opción está activa (`activeItem`) y avisa al padre cuando se selecciona otra (`onSelect`). A esto se le llama **elevar el estado** (lifting state up).

8.3 DashboardLayout

Este es el componente clave de la reutilización. Arma el esqueleto (Navbar + Sidebar + área de contenido) y coloca en el centro lo que reciba como `children`.

src/components/layout/DashboardLayout.jsx

```
import Navbar from './Navbar';
import Sidebar from './Sidebar';

const DashboardLayout = ({ user, onLogout, menuItems, activeItem, onSelect, children }) => {
  return (
    <div className="dashboard">
      <Navbar user={user} onLogout={onLogout} />
      <div className="dashboard-body">
        <Sidebar items={menuItems} activeItem={activeItem} onSelect={onSelect} />
        <main className="dashboard-content">{children}</main>
      </div>
    </div>
  );
};

export default DashboardLayout;
```

9. Parte 6: Componentes de visualización

9.1 StatCard

Tarjeta que muestra un número grande con una descripción. El color se aplica armando el nombre de la clase CSS con un **template literal**.

src/components/dashboard/StatCard.jsx

```
const StatCard = ({ label, value, color = 'blue' }) => {
  return (
    <div className={`stat-card stat-${color}`}>
      <span className="stat-value">{value}</span>
      <span className="stat-label">{label}</span>
    </div>
  );
};

export default StatCard;
```

9.2 DataTable

Una tabla **genérica**: sirve para usuarios, cursos o cualquier otra lista. Recibe dos props:

- `columns`: arreglo que indica qué campo mostrar (`key`) y con qué encabezado (`header`).
- `rows`: arreglo de objetos con los datos. Cada objeto debe tener un `id`.

src/components/dashboard/DataTable.jsx

```
const DataTable = ({ columns, rows }) => {
  return (
    <table className="data-table">
      <thead>
        <tr>
          {columns.map((col) => (
            <th key={col.key}>{col.header}</th>
          ))}
        </tr>
      </thead>
      <tbody>
        {rows.map((row) => (
          <tr key={row.id}>
            {columns.map((col) => (
              <td key={col.key}>{row[col.key]}</td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  );
};

export default DataTable;
```

Como la tabla solo muestra las columnas indicadas, aunque el objeto de usuario tenga el campo `password`, **nunca se muestra**. Tú decides qué información exponer.

10. Parte 7: Páginas

Las páginas combinan los componentes anteriores y contienen la lógica de cada pantalla.

10.1 LoginPage

Maneja tres estados: correo, contraseña y mensaje de error. Al enviar el formulario busca un usuario que coincida con `find()`. Si lo encuentra, llama a la función `onLogin` que recibió de `App`.

src/pages/LoginPage.jsx

```
import { useState } from 'react';
import Card from '../components/common/Card';
import InputField from '../components/common/InputField';
import Button from '../components/common/Button';
import { users } from '../data/users';

const LoginPage = ({ onLogin }) => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = (event) => {
    event.preventDefault(); // evita que la página se recargue
    const found = users.find((u) => u.email === email && u.password === password);

    if (found) {
      setError('');
      onLogin(found);
```

```
    } else {
      setError('Correo o contraseña incorrectos');
    }
  };

  return (
    <div className="login-page">
      <Card title="Iniciar sesión">
        <form onSubmit={handleSubmit}>
          <InputField
            label="Correo"
            type="email"
            name="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            placeholder="usuario@escuela.edu"
          />
          <InputField
            label="Contraseña"
            type="password"
            name="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            placeholder="********"
          />
          {error && <p className="error">{error}</p>}
          <Button type="submit" fullWidth>
            Entrar
          </Button>
        </form>
      </Card>
    </div>
  );
};

export default LoginPage;
```

`event.preventDefault()` evita que el navegador recargue la página al enviar el formulario, que es el comportamiento normal de HTML. Usar `<form>` con `type="submit"` permite además entrar presionando la tecla Enter.

10.2 AdminDashboard

Usa `DashboardLayout` y cambia el contenido del centro según la sección seleccionada en el menú. Las columnas de las tablas se definen fuera del componente porque no cambian.

src/pages/AdminDashboard.jsx

```
import { useState } from 'react';
import DashboardLayout from '../components/layout/DashboardLayout';
import Card from '../components/common/Card';
import StatCard from '../components/dashboard/StatCard';
import DataTable from '../components/dashboard/DataTable';
import { users, courses } from '../data/users';

const menuItems = ['Resumen', 'Usuarios', 'Cursos'];

const userColumns = [
  { key: 'id', header: 'ID' },
```

```
  { key: 'name', header: 'Nombre' },
  { key: 'email', header: 'Correo' },
  { key: 'role', header: 'Rol' },
];

const courseColumns = [
  { key: 'id', header: 'ID' },
  { key: 'name', header: 'Curso' },
  { key: 'progress', header: 'Avance (%)' },
];

const AdminDashboard = ({ user, onLogout }) => {
  const [section, setSection] = useState('Resumen');
  const totalAdmins = users.filter((u) => u.role === 'admin').length;

  return (
    <DashboardLayout
      user={user}
      onLogout={onLogout}
      menuItems={menuItems}
      activeItem={section}
      onSelect={setSection}
    >
      <h2>Panel de Administración</h2>

      {section === 'Resumen' && (
        <div className="stats-grid">
          <StatCard label="Usuarios registrados" value={users.length} color="blue" />
          <StatCard label="Administradores" value={totalAdmins} color="purple" />
          <StatCard label="Cursos activos" value={courses.length} color="green" />
        </div>
      )}

      {section === 'Usuarios' && (
        <Card title="Lista de usuarios">
          <DataTable columns={userColumns} rows={users} />
        </Card>
      )}

      {section === 'Cursos' && (
        <Card title="Cursos">
          <DataTable columns={courseColumns} rows={courses} />
        </Card>
      )}
    </DashboardLayout>
  );
};

export default AdminDashboard;
```

10.3 UserDashboard

Compárala con la página del administrador: **usa exactamente los mismos componentes**, pero con otro menú, otros datos y otras columnas. Aquí se aprecia el beneficio de la reutilización.

src/pages/UserDashboard.jsx

```
import { useState } from 'react';
import DashboardLayout from '../components/layout/DashboardLayout';
```

```
import Card from '../components/common/Card';
import StatCard from '../components/dashboard/StatCard';
import DataTable from '../components/dashboard/DataTable';
import { courses } from '../data/users';

const menuItems = ['Inicio', 'Mis cursos', 'Mi perfil'];

const courseColumns = [
  { key: 'name', header: 'Curso' },
  { key: 'progress', header: 'Avance (%)' },
];

const UserDashboard = ({ user, onLogout }) => {
  const [section, setSection] = useState('Inicio');
  const total = courses.reduce((sum, c) => sum + c.progress, 0);
  const average = Math.round(total / courses.length);

  return (
    <DashboardLayout
      user={user}
      onLogout={onLogout}
      menuItems={menuItems}
      activeItem={section}
      onSelect={setSection}
    >
      <h2>Bienvenido, {user.name}</h2>

      {section === 'Inicio' && (
        <div className="stats-grid">
          <StatCard label="Cursos inscritos" value={courses.length} color="blue" />
          <StatCard label="Avance promedio" value={`${average}%`} color="green" />
        </div>
      )}

      {section === 'Mis cursos' && (
        <Card title="Mis cursos">
          <DataTable columns={courseColumns} rows={courses} />
        </Card>
      )}

      {section === 'Mi perfil' && (
        <Card title="Mi perfil">
          <p><strong>Nombre:</strong> {user.name}</p>
          <p><strong>Correo:</strong> {user.email}</p>
          <p><strong>Rol:</strong> Usuario registrado</p>
        </Card>
      )}
    </DashboardLayout>
  );
};

export default UserDashboard;
```

`reduce()` recorre el arreglo acumulando la suma de los avances; después se divide entre el número de cursos para obtener el promedio.

11. Parte 8: App.jsx y estilos

11.1 App.jsx

`App` es el componente raíz. Guarda al usuario que inició sesión y decide qué página mostrar:

- Si no hay usuario (`null`) → muestra el Login.
- Si el rol es `admin` → muestra el dashboard de administrador.
- En cualquier otro caso → muestra el dashboard de usuario.

src/App.jsx

```
import { useState } from 'react';
import LoginPage from './pages/LoginPage';
import AdminDashboard from './pages/AdminDashboard';
import UserDashboard from './pages/UserDashboard';

function App() {
  const [currentUser, setCurrentUser] = useState(null);

  const handleLogin = (user) => setCurrentUser(user);
  const handleLogout = () => setCurrentUser(null);

  if (!currentUser) {
    return <LoginPage onLogin={handleLogin} />;
  }

  if (currentUser.role === 'admin') {
    return <AdminDashboard user={currentUser} onLogout={handleLogout} />;
  }

  return <UserDashboard user={currentUser} onLogout={handleLogout} />;
}

export default App;
```

11.2 Estilos

Reemplaza el contenido de `src/index.css` con lo siguiente. Los nombres de las clases coinciden con los `className` usados en los componentes.

src/index.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: system-ui, sans-serif; background: #f1f5f9; color: #1e293b; }

/* ===== Botones ===== */
.btn { padding: 0.6rem 1.2rem; border: none; border-radius: 6px; font-size: 0.95rem; cursor:
pointer; }
.btn-primary { background: #2563eb; color: white; }
.btn-primary:hover { background: #1d4ed8; }
.btn-secondary { background: #e2e8f0; color: #1e293b; }
.btn-danger { background: #dc2626; color: white; }
.btn-full { width: 100%; }

/* ===== Campos de texto ===== */
.input-field { display: flex; flex-direction: column; margin-bottom: 1rem; }
.input-field label { font-size: 0.9rem; font-weight: 600; margin-bottom: 0.3rem; }
.input-field input { padding: 0.6rem; border: 1px solid #cbd5e1; border-radius: 6px; font-
size: 0.95rem; }

/* ===== Tarjeta ===== */
```

```
.card { background: white; border-radius: 10px; padding: 1.5rem; box-shadow: 0 2px 8px
rgba(0,0,0,0.08); }
.card-title { margin-bottom: 1rem; }

/* ===== Login ===== */
.login-page { min-height: 100vh; display: flex; align-items: center; justify-content: center;
padding: 1rem; }
.login-page .card { width: 100%; max-width: 380px; }
.error { color: #dc2626; font-size: 0.9rem; margin-bottom: 1rem; }

/* ===== Navbar ===== */
.navbar { display: flex; justify-content: space-between; align-items: center; padding: 1rem
1.5rem; background: #1e293b; color: white; }
.navbar-brand { font-size: 1.2rem; font-weight: 700; }
.navbar-user { display: flex; align-items: center; gap: 1rem; }

/* ===== Layout del dashboard ===== */
.dashboard-body { display: flex; min-height: calc(100vh - 64px); }
.sidebar { width: 200px; background: white; border-right: 1px solid #e2e8f0; padding: 1rem 0;
}
.sidebar ul { list-style: none; }
.sidebar button { width: 100%; text-align: left; padding: 0.75rem 1.5rem; background: none;
border: none; font-size: 0.95rem; cursor: pointer; }
.sidebar button:hover { background: #f1f5f9; }
.sidebar button.active { background: #dbeafe; color: #1d4ed8; font-weight: 600; }
.dashboard-content { flex: 1; padding: 2rem; }
.dashboard-content h2 { margin-bottom: 1.5rem; }

/* ===== Tarjetas de estadísticas ===== */
.stats-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap:
1rem; }
.stat-card { display: flex; flex-direction: column; background: white; border-radius: 10px;
padding: 1.5rem; border-left: 5px solid; box-shadow: 0 2px 8px rgba(0,0,0,0.08); }
.stat-blue { border-color: #2563eb; }
.stat-green { border-color: #16a34a; }
.stat-purple { border-color: #7c3aed; }
.stat-value { font-size: 2rem; font-weight: 700; }
.stat-label { color: #64748b; font-size: 0.9rem; }

/* ===== Tabla ===== */
.data-table { width: 100%; border-collapse: collapse; }
.data-table th, .data-table td { padding: 0.75rem; text-align: left; border-bottom: 1px solid
#e2e8f0; }
.data-table th { background: #f8fafc; font-weight: 600; }
```

12. Parte 9: Pruebas

Con el servidor corriendo (`npm run dev`), realiza cada prueba y marca el resultado:

#	Acción	Resultado esperado	✓
1	Entrar con admin@escuela.edu / admin123	Se muestra el Panel de Administración	
2	Entrar con luis@escuela.edu / user123	Se muestra “Bienvenido, Luis Pérez”	

#	Acción	Resultado esperado	✓
3	Entrar con una contraseña incorrecta	Aparece el mensaje de error en rojo	
4	En el admin, dar clic en “Usuarios”	Aparece la tabla con 3 usuarios, sin contraseñas	
5	En el usuario, dar clic en “Mi perfil”	Se muestran nombre, correo y rol	
6	Presionar “Cerrar sesión”	Regresa al Login con los campos vacíos	
7	Escribir usuario y contraseña y presionar Enter	Inicia sesión sin dar clic en el botón	

> **Si algo no funciona**
> Abre la consola del navegador (F12 o Cmd + Option + I en Mac). Los errores más comunes son: una ruta de `import` mal escrita, un componente al que le falta `export default`, o un nombre de archivo con mayúsculas distintas al del import.

13. Retos adicionales

Resuelve al menos **dos** de los siguientes retos. Todos se pueden hacer reutilizando los componentes existentes.

1. **Nuevo usuario:** agrega un cuarto usuario con rol `user` en `users.js` y comprueba que la estadística del admin se actualiza sola.
2. **Botón de peligro:** usa `variant="danger"` para el botón de “Cerrar sesión” en el dashboard de administrador únicamente. Pista: agrega una prop a `Navbar`.
3. **Barra de progreso:** crea un componente `ProgressBar` que reciba un porcentaje y úsalo en la sección “Mi perfil” para mostrar el avance promedio.
4. **Mostrar contraseña:** en el Login agrega un botón que alterne el tipo del campo entre `password` y `text`.
5. **Buscador:** en la sección “Usuarios” del admin agrega un `InputField` que filtre la tabla por nombre con `filter()`.
6. **Tercer rol:** agrega el rol `teacher` con su propio dashboard, reutilizando `DashboardLayout`.

14. Preguntas de reflexión

Responde en tu reporte:

1. ¿Qué ventaja tiene que `DataTable` reciba las columnas como prop en lugar de tenerlas escritas dentro del componente?
2. ¿Por qué el estado `section` vive en cada página y no dentro de `Sidebar`?

3. ¿Qué es la prop `children` y en qué componentes de este laboratorio se usó?
4. ¿Qué pasaría si quitas `event.preventDefault()` del Login? Pruébalo y describe lo que observas.
5. Explica por qué no es seguro validar contraseñas en el FrontEnd. ¿Dónde debería hacerse?

15. Entregables y evaluación

15.1 Entregables

- Carpeta del proyecto comprimida **sin** la carpeta `node_modules`.
- Reporte en PDF con capturas de pantalla de las tres pantallas, la tabla de pruebas llena, los retos resueltos y las respuestas a las preguntas de reflexión.

15.2 Rúbrica

Criterio	Descripción	Puntos
Estructura del proyecto	Carpetas y archivos organizados como en la Parte 2; nombres con la convención correcta.	15
Componentes reutilizables	Los componentes comunes, de layout y de dashboard funcionan y se reutilizan en ambas páginas.	25
Login y roles	El login valida credenciales, muestra errores y dirige a cada rol a su dashboard.	20
Navegación y estado	El menú lateral cambia de sección y el cierre de sesión funciona.	10
Retos adicionales	Al menos dos retos resueltos correctamente.	15
Reporte	Capturas, tabla de pruebas y respuestas claras a las preguntas de reflexión.	15
Total		**100**